GDD e Especificação Técnica: Projeto "Muck 3D" (Unity 6 / C#)

---

1. Visão Geral do Jogo e Pilares de Design

O projeto consiste em um jogo 3D de sobrevivência com fortes elementos de *Roguelike*, combate em primeira pessoa (*hack-and-slash/action*), geração procedural de terreno e ciclo acelerado de dia/noite com escalonamento de dificuldade.

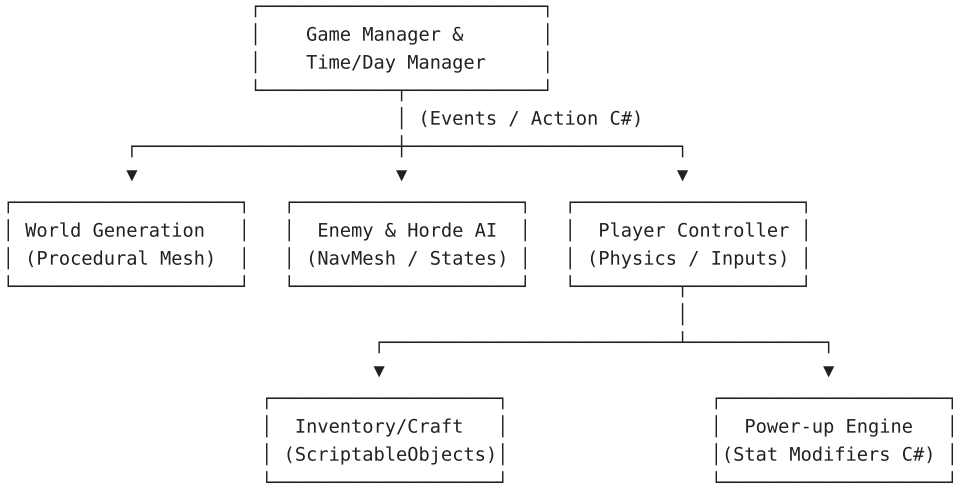
Pilares Fundamentais

1. **Ritmo Acelerado (*Arcade Survival*):** Diferente de simuladores lentos, a coleta de madeira, minério e recursos é dinâmica, com movimentação fluida (estilo *Source Engine/Quake* com pulo, aceleração e deslizamento).
2. **Escala Roguelike Exponencial:** Baús espalhados concedem melhorias cumulativas (*power-ups* que aumentam velocidade, dano crítico, ataque em cadeia, etc.), transformando o jogador de vulnerável a uma força imparável.
3. **Pressão Constante de Tempo:** Cada dia concede tempo seguro para exploração e mineração; cada noite traz hordas agressivas e aparição de chefes periódicos.
4. **Arquitetura Desacoplada e Pronta para Rede:** Toda a lógica de regras de jogo e estado é desacoplada da camada visual e de entrada, facilitando a inclusão futura de arquitetura cliente-servidor (*Netcode for GameObjects* ou *Fish-Net*).

---

2. Arquitetura de Software Geral (Unity 6 / C#)

Como desenvolvedor, manter a base de código limpa evitará o débito técnico clássico de desenvolvimento de jogos (como *God MonoBehaviours* e dependências circulares).



Princípios Técnicos:

- **Data-Driven com `ScriptableObject`:** Itens, receitas, atributos de inimigos, biomas e *power-ups* são tratados como dados serializados, desacoplados dos componentes de cena.
- **Stat Modifier System:** Atributos do jogador e dos monstros usam um padrão de composição onde o valor base é multiplicado ou somado por modificadores ativos:

$$\text{ValorFinal} = (\text{Base} + \sum \text{FlatModifiers}) \times (1 + \sum \text{PercentModifiers})$$

- **Event-Driven Communication:** Comunicação entre módulos via `System.Action` C# nativas ou *ScriptableObject Events*, minimizando chamadas acopladas de `GetComponent<T>()` durante o *game loop*.
- **Abstração de Input:** Uso estrito do novo **Unity Input System** com mapeamento de ações desacoplado da lógica de execução.

---

3. Módulo 1: Geração de Mundo Procedural

O mundo de *Muck* é uma ilha cercada por água com relevo montanhoso, cavernas rasas e distribuição esparsa de recursos (árvores, rochas, flores, baús e altares).

3.1. Geração da Malha de Terreno (Procedural Mesh)

1. **Grid de Chunks:** A ilha é dividida em blocos de malha (ex: 16×16 ou 32×32 vértices por chunk).
2. **Função de Altura com Perlin/Simplex Noise:**

$$H(x, z) = \left(\sum_{i=0}^{n-1} \text{amplitude}_i \cdot \text{Noise}(x \cdot \text{freq}_i, z \cdot \text{freq}_i) \right) \times \text{Falloff}(x, z)$$

   - **Mascara de Falloff:** Reduz a altura conforme a distância do centro $(x, z) = (0, 0)$ aumenta, garantindo que as bordas da ilha afundem no nível do mar ($H \leq 0$).
3. **Cálculo de Normais e Tangentes:** Normais calculadas dinamicamente via produto vetorial dos triângulos da malha para suporte a shaders estilizados de iluminação.

3.2. Posicionamento de Elementos (Poisson Disc Sampling)

Para evitar que pedras, árvores e baús nasçam uns sobre os outros:

- **Algoritmo de Poisson Disc Sampling 2D:** Gera coordenadas candidatas com distância mínima de corte r.
- **Raycast Vertical:** Da coordenada calculada $(x, Y_{\max}, z)$, faz-se um raycast para baixo contra o *Mesh Collider* do terreno gerado para determinar a altura exata y e a inclinação da superfície (normal).
- **Filtros de Altura e Declive:**
  - Árvores e grama: Altura entre nível do mar ($y > 2$) e montanha média ($y < 25$), inclinação máxima de 30°.
  - Minérios avançados (Ouro, Adamantite, Mitril): Áreas elevadas ($y > 25$) ou cavernas.
  - Baús e Totens: Superfícies planas ($< 15^\circ$).

---

4. Módulo 2: Movimentação do Jogador e Física

Muck é conhecido pela movimentação fluida inspirada em jogos clássicos com *bunny hopping* e sensação de inércia.

4.1. Escolha de Arquitetura: `Rigidbody` Customizado

- Evitar o `CharacterController` nativo padrão se o objetivo for simulação dinâmica de aceleração, atrito no solo e deslize em declives.
- Utiliza-se um `Rigidbody` com interpolação ativada e `FreezeRotation` em X e Z.

4.2. Vetores de Aceleração e Atrito

- **No Solo:** Aceleração linear com atrito dinâmico.
- **No Ar:** Redução do controle direcional, mas preservação da quantidade de movimento (permitindo saltos sucessivos mantendo ou ganhando velocidade linear).
- **Slide (Deslize):** Ao pressionar agachar durante corrida, converte a direção atual em um impulso tangencial à inclinação do solo, reduzindo temporariamente a colisão do jogador pela metade da altura.

---

5. Módulo 3: Coleta, Inventário e Crafting

5.1. Sistema de Recursos e Ferramentas

- **Resistência do Objeto:** Cada recurso no mundo possui:
  - `RequiredToolType` (Ex: Machado, Picareta).
  - `Tier` (Madeira = 0, Pedra = 1, Ferro = 2, Ouro = 3, Mitril = 4, Adamantite = 5).
  - `Health` (Pontos de vida da entidade antes de dropar itens).

- **Interação e Dano ao Recurso:** Caso o jogador acerte o objeto com ferramenta de *Tier* igual ou superior, aplica o dano da ferramenta. Caso contrário, causa dano residual (ex: 1 de dano) e toca som de impacto metálico/bloqueio.

5.2. Inventário e Slots

- **Modelo de Dados:**

```
[System.Serializable]
public class InventorySlot {
    public ItemData item;
    public int amount;
    public bool IsEmpty => item == null || amount <= 0;
}
```

- **Hotbar (0 a 4):** Acesso rápido mapeado nas teclas numéricas do teclado.
- **Crafting:**
    - *Inventário Básico:* Receitas simples de sobrevivência (bancada de trabalho, ferramentas primitivas).
    - *Estações de Trabalho (Workbenches, Fornalha, Bigorna):* Ativam menus dedicados quando o jogador está em um raio de interação (2.5m).

---

6. Módulo 4: Combate, Dano e Hitbox

6.1. Detecção de Dano (Hybrid Overlap Volume)

Para garantir acertos responsivos sem o custo computacional ou instabilidade de colisores contínuos em alta velocidade:

1. Ao iniciar o ataque, a animação aciona um gatilho (*Animation Event*).
2. Durante a janela de ataque (ex: entre 30% e 60% da animação), executa-se periodicamente uma consulta de colisão: `Physics.OverlapSphereNonAlloc` ou `Physics.OverlapCapsuleNonAlloc` à frente do jogador.
3. Garante-se uma lista temporária `HashSet<IDamageable>` por golpe para que um único golpe não atinja o mesmo alvo múltiplas vezes na mesma animação.

6.2. Feedback e Sensação de Impacto (*Game Juice*)

- **Hitstop:** Congelamento de microssegundos (0.03s) na escala de tempo ao acertar um golpe crítico.
- **Floating Damage Numbers:** Números dinâmicos saltando na tela (cor branca para dano comum, amarela para dano crítico, vermelha para dano sofrido).
- **Knockback Vector:**

$$\vec{F}_{\text{knockback}} = (\vec{D}_{\text{atacante} \to \text{alvo}} + \vec{u}_y \cdot 0.3) \times \text{Força}$$

---

7. Módulo 5: Sistema Roguelike de Power-ups

O coração da progressão de curto prazo de *Muck* são os itens passivos acumulativos (*stacks*).

7.1. Arquitetura Modular do Power-up

```
public abstract class PowerupEffectSO : ScriptableObject {
    public string powerupId;
    public string displayName;
    public Rarity rarity; // Comum, Raro, Lendário
    public Sprite icon;

    public abstract void OnApply(PlayerStats stats, int currentStacks);
    public abstract void OnHit(PlayerStats attacker, IDamageable victim, ref DamagePayload payload, int stacks
    public abstract void OnKill(PlayerStats attacker, IDamageable victim, int stacks);
}
```

7.2. Exemplos de Power-ups para a Tabela Base

Nome do Item	Raridade	Efeito por Stack
Sneakers	Comum	$+10\%$ Velocidade de Movimento base.
Peanut Butter	Comum	$+15\%$ Regeneração de Estamina.
Dumbbell	Raro	$+8\%$ Dano Físico Base em todas as armas.
Horseshoe	Raro	$+10\%$ Chance de Acerto Crítico.
Crimson Dagger	Raro	Cura 5% do dano causado no acerto crítico.
Knockback Ball	Raro	$+25\%$ Força de repulsão aplicada em inimigos.
Juice	Lendário	Cada acerto aumenta a velocidade de ataque temporariamente em $+10\%$ (acumula até 5 vezes).
Sniper Scope	Lendário	2% de chance de causar $10\times$ de dano.

7.3. Sistema de Baús

- **Baú Grátis / Baú Inicial:** Contém 1 item comum.
- **Baús Pagos com Moedas (Gold Chests):**
  - O jogador adquire moedas matando inimigos.
  - Baús espalhados exigem moedas para abrir (25, 50, 100 moedas), escalando o preço com o passar dos dias.

---

8. Módulo 6: Ciclo Dia/Noite e Escalonamento Matemático

O tempo no jogo é medido em dias. Cada dia dura um tempo fixo (ex: 180s dia / 120s noite).

8.1. Fórmulas de Escala de Dificuldade

Seja D o número do dia atual ($D \geq 1$):

1. **Multiplicador de Vida dos Inimigos:**

$$M_{\text{HP}}(D) = 1.0 + 0.35 \times (D - 1)^{1.25}$$

2. **Multiplicador de Dano dos Inimigos:**

$$M_{\text{DMG}}(D) = 1.0 + 0.20 \times (D - 1)$$

3. **Orçamento de Spawn por Onda Noturna ($Budget$):**

$$\text{Budget}(D) = 50 + 30 \times D^{1.4}$$

   - Cada inimigo possui um custo de spawn (ex: Goblins fracos = 10pts, Golens de Pedra = 40pts, Chefes = 200pts).
   - O spawner noturno gasta o orçamento sorteando inimigos em um raio ao redor do jogador, fora do campo de visão direto.

8.2. Gatilhos de Bosses

- **Noite 3 a 5:** Surgimento obrigatório do primeiro Chefe (ex: *Big Chunk* - Golem de pedra com ataque de salto e arremesso de pedras).
- **Noite 6+:** Surgimento de múltiplos chefes ou chefes ágeis (ex: *Gronk* - guerreiro bípede veloz com ataque de lâminas duplas).
- **Altar de Totem:** Permite que o jogador force o combate contra um chefe a qualquer momento durante o dia em troca de recompensas raras garantidas.

---

9. Módulo 7: Inteligência Artificial (Inimigos e Bosses)

9.1. Sistema de Navegação (Dynamic NavMesh)

- Na Unity 6, utiliza-se o pacote **NavMesh Building Components** para assar a malha de navegação (`NavMeshSurface.BuildNavMesh()`) em tempo de execução assim que a geração procedural do terreno terminar.

9.2. Máquina de Estados (Finite State Machine / FSM)

Cada inimigo implementa estados desacoplados:

- `Idle / Patrol`: Ronda lenta pelas proximidades.
- `Chase`: Localiza o jogador e se move diretamente na velocidade máxima calculada.
- `Attack`: Ao atingir a distância de ataque, para e executa o golpe.
- `Stagger / Knockback`: Desativa o agente NavMesh momentaneamente e aplica força física pelo Rigidbody quando toma golpes pesados.

---

10. Roadmap de Implementação Modular (Guia para Gemini CLI)

Para trabalhar de forma produtiva com o Gemini CLI (Antigravity), o desenvolvimento deve ocorrer em módulos pequenos, independentes e testáveis.

```
┌──────────────────────────────────────────────────────┐
│ FASE 1: Player Controller & Física 3D                │
├──────────────────────────────────────────────────────┤
│ FASE 2: Geração de Terreno e Recursos Procedurais    │
├──────────────────────────────────────────────────────┤
│ FASE 3: Coleta, Mineração e Inventário               │
├──────────────────────────────────────────────────────┤
│ FASE 4: Combate e Sistema de Inimigos Básicos        │
├──────────────────────────────────────────────────────┤
│ FASE 5: Power-ups Roguelike e Baús                   │
├──────────────────────────────────────────────────────┤
│ FASE 6: Ciclo Dia/Noite, Spawner de Hordas e Bosses  │
├──────────────────────────────────────────────────────┤
│ FASE 7: Refinamento, Salve/Estado e Preparação Netcode │
└──────────────────────────────────────────────────────┘
```

Prompt Templates Prontos para Alimentar a CLI:

Etapa 1: Controle de Personagem

> *"Atue como arquiteto Unity C#. Crie um `PlayerController` baseado em `Rigidbody` usando o novo Input System da Unity 6. Deve conter movimentação fluida em primeira pessoa, aceleração no solo, preservação de momento no ar para bunny hopping, câmera desacoplada (look script com clamping vertical) e suporte a deslizamento (slide). Documente as variáveis no Inspector."*

Etapa 2: Geração de Terreno

> *"Gere um script `ProceduralIslandGenerator` em C# para Unity 6. Ele deve gerar uma malha procedural (MeshFilter e MeshCollider) com Perlin Noise e uma máscara de Falloff radial para criar uma ilha circular cercada por água. Permita configurar semente (Seed), resolução, frequência, amplitude e tamanho do chunk."*

Etapa 3: Power-up Engine

> *"Implemente a arquitetura do sistema de Power-ups usando ScriptableObjects e Stat Modifiers. Crie a classe abstrata `PowerupEffectSO`, o container `PlayerStats` (vida, velocidade, dano, crítico) e três exemplos de itens: um que aumenta velocidade percentual, um que aumenta chance de crítico e um que aplica lifesteal."*

---

11. Preparação para o Multiplayer (Passo Futuro)

Para evitar reescrever todo o projeto quando decidir incluir suporte a rede:

1. **Evite depender de referências locais diretas:** Em vez de acessar diretamente `player.transform.position`, organize a lógica em torno de identificadores de entidade (`EntityID`).
2. **Separe a Lógica da Representação Visual:**
   - Crie scripts lógicos (ex: `HealthSystem`, `CombatSystem`) puros, que disparam eventos.
   - Crie scripts visuais (ex: `PlayerAnimations`, `DamageEffectsView`) que apenas escutam esses eventos para reproduzir partículas, sons e animações.

3. **Deterministic Seed:** A semente (*Seed*) da geração de mundo deve ser sincronizada entre todos os clientes no início para que cada máquina gere exatamente o mesmo mapa procedural sem necessidade de trafegar dados pesados de malha pela rede.

---

12. Ferramentas e Ecossistema de IA Recomendados

Para acelerar o desenvolvimento solo e criar novos biomas, armas, monstros e chefes sem gargalos em modelagem manual:

12.1. Modelagem 3D, Rigging e Animação

1. **Blender:**
   - A ferramenta essencial e gratuita para criar modelos *Low Poly* (técnica com paleta de cores/UV colapsada, exatamente como o Muck original).
   - Modificadores como *Decimate* permitem reduzir a contagem de polígonos instantaneamente.
2. **Adobe Mixamo:**
   - Plataforma gratuita de *auto-rigging* e catálogo com centenas de animações (andar, correr, socar, golpear com espadas, morrer, ataques de monstros bípedes).
   - Você sobe seu personagem/monstro em formato FBX ou OBJ sem esqueleto, marca onde ficam queixo, pulsos e joelhos, e obtém um modelo animado pronto para a Unity.
3. **Cascalho / Blockbench:**
   - Alternativa levíssima ao Blender para criar modelos com geometria simples e visual estilizado de blocos ou *low poly*.

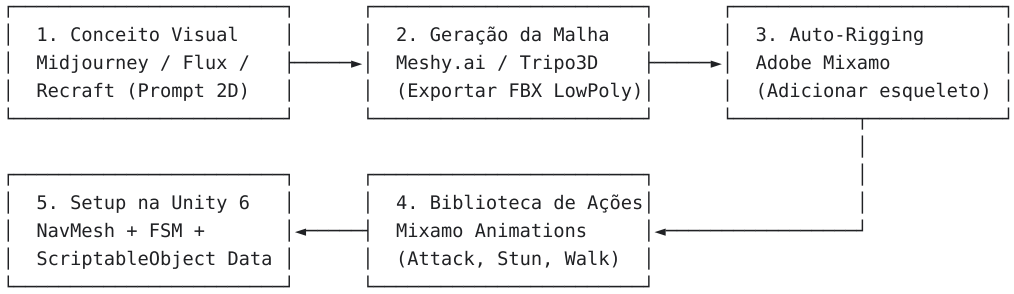
12.2. Ferramentas de IA para Geração 3D (Text-to-3D / Image-to-3D)

Atualmente existem ferramentas de IA bastante maduras para gerar objetos e monstros rapidamente:

- **Meshy.ai:** Uma das melhores ferramentas atuais para *Text-to-3D* e *Image-to-3D*. Possui modo específico para **Low Poly / Game Asset**, gerando modelos limpos em formato `.fbx` com texturas e até auto-rigging básico. Excelente para criar baús, totens, rochas mágicas e armas.
- **Tripo3D:** Gera modelos 3D rápidos em segundos a partir de texto ou imagens conceituais. Permite exportar malhas simplificadas ideais para protótipos de chefes e inimigos.
- **Rodin (Deemos Tech) / Luma Genie:** Úteis para gerar assets orgânicos complexos (como árvores fantásticas e estátuas de chefes) que podem ser otimizados no Blender via *Decimate*.
- **Unity Muse:** Conjunto oficial de IAs da própria Unity (integrado ao editor) para gerar texturas PBR, sprites e prototipar lógicas de animação diretamente na engine.

12.3. Pipeline de Criação de Bosses / Inimigos com IA

Para criar um novo Boss (ex: "Ent Ancião" ou "Golem de Lava") em menos de 1 hora:



12.4. Áudio, Efeitos Sonoros e Músicas

- **jsfxr / sfxr:** Gerador clássico de efeitos sonoros 8-bit e arcade (pulos, cliques, explosões, acertos).
- **ElevenLabs Sound Effects:** IA que gera efeitos sonoros realistas ou estilizados a partir de comandos textuais (ex: *"heavy stone golem footstep on grass"*, *"wood chopping with axe hit"*).
- **Suno / Udio:** Geração de trilhas sonoras orquestrais ou dinâmicas para batalhas de chefes e exploração diurna.

12.5. Assets Prontos Recomendados (Unity Asset Store & Itch.io)

- **KayKit (Kay Lousberg - itch.io):** Pacotes gratuitos de alta qualidade em Low Poly (monstros, masmorras, armas e sobrevivência), sob licença CC0 (domínio público).

- **Synty Studios (Polygon Series):** Padrão da indústria para protótipos e jogos comerciais Low Poly.
- **Kenney.nl ("Asset Jesus"):** Centenas de modelos 3D gratuitos, texturas de interface e efeitos sonoros livres para qualquer uso.